

Practical : 9	Implementation of Prim's Algorithm for Minimum Spanning Tree.
Date : 12/08/2026	

Objective :- To implement Prim's Algorithm in C++ using an adjacency-matrix representation to construct the Minimum Spanning Tree (MST) of a weighted, connected, undirected graph, and to analyse its time and space complexity.

Program :-

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

const int INF = INT_MAX;

// ----- Prim's Algorithm (Adjacency Matrix,  $O(V^2)$ ) -----
int primMST(vector<vector<int>>& graph, int V, vector<int>& parent) {
    vector<int> key(V, INF);
    vector<bool> inMST(V, false);
    parent.assign(V, -1);
    key[0] = 0; // start building the MST from vertex 0

    for (int count = 0; count < V; count++) {
        // pick the un-visited vertex with the smallest key value
        int u = -1, minKey = INF;
        for (int v = 0; v < V; v++) {
            if (!inMST[v] && key[v] < minKey) {
                minKey = key[v];
                u = v;
            }
        }

        inMST[u] = true; // finalise u into the MST

        // relax the keys of all vertices adjacent to u
        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !inMST[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }

    int totalWeight = 0;
    for (int i = 1; i < V; i++) totalWeight += key[i];
    return totalWeight;
}

int main() {
```

```
int V = 5;
vector<vector<int>> graph = {
{0, 2, 0, 6, 0},
{2, 0, 3, 8, 5},
{0, 3, 0, 0, 7},
{6, 8, 0, 0, 9},
{0, 5, 7, 9, 0}
};

vector<int> parent;
int totalWeight = primMST(graph, V, parent);

cout << "Edge \tWeight\n";
for (int i = 1; i < V; i++)
cout << parent[i] << " - " << i << "\t" << graph[parent[i]][i] << "\n";

cout << "Total weight of MST: " << totalWeight << endl;

return 0;
}
```

Output :-

```
Edge      Weight
0 - 1      2
1 - 2      3
0 - 3      6
1 - 4      5
Total weight of MST: 16
```

Analysis:-

Time & Space Complexity (Adjacency Matrix implementation, V = vertices, E = edges):

Method	Time Complexity	Space Complexity
Prim's - Adjacency Matrix	$O(V^2)$	$O(V^2)$
Prim's - Adjacency List + Min-Heap	$O(E \log V)$	$O(V + E)$

Prim's algorithm grows the MST one vertex at a time, always adding the cheapest edge that connects the tree to a new vertex. With an adjacency matrix, each of the V iterations scans all V vertices twice — once to find the minimum key and once to relax neighbours — giving $O(V^2)$ time, and the matrix itself costs $O(V^2)$ space; this is efficient for dense graphs. Using an adjacency list with a binary min-heap instead reduces the extract-min and decrease-key work to $O(\log V)$ each, so the total time falls to $O(E \log V)$, with $O(V + E)$ space — the better choice for sparse graphs. For the 5-vertex sample graph above, the algorithm correctly builds an MST of total weight 16 by repeatedly selecting the smallest-weight edge crossing the cut between the visited and unvisited vertices, which is exactly the cut property Prim's algorithm relies on for correctness.

Conclusion :- Prim's Algorithm was successfully implemented in C++ using an adjacency-matrix representation to compute the Minimum Spanning Tree of a weighted undirected graph. The program correctly identified the MST edges and total weight for the sample graph, confirming the $O(V^2)$ time and $O(V^2)$ space behaviour of the matrix-based approach. For larger, sparser graphs an adjacency-list with a min-heap would be preferred, reducing the time complexity to $O(E \log V)$ while keeping space proportional to $O(V + E)$.